AI DZ

Academy

Mastering Machine Learning

A Comprehensive Course

From Fundamentals to Advanced Implementation

Instructors:
Abdelhak Samir D. (PhD in Computer Science and Automatic Control, University of Lille)

Certification: Upon successful completion, students receive AI DZ Academy's Professional Certificate in Machine Learnin

Prerequisites: Python programming, basic calculus, linear algebra

Duration: 4 weeks (12 hours of lectures and practical work)

Contents

Course Overview **3**

1 The Machine Learning Landscape **4**
1.1 What is Machine Learning? . . . 4
1.2 A Brief History of Machine Learning . . . 4
1.3 Where Do We Use Machine Learning? . . . 4
1.3.1 Real Examples That Help People . . . 4

2 The Three Main Types of Machine Learning **5**
2.1 Supervised Learning: Learning with a Teacher . . . 5
2.1.1 Common Tasks in Supervised Learning . . . 5
2.2 Unsupervised Learning: Finding Hidden Patterns . . . 5
2.3 Reinforcement Learning: Learning by Trial and Error . . . 6
2.4 The Three Magic Steps of Learning . . . 8

3 PyTorch Mastery: From Fundamentals to First Model **10**
3.1 What is PyTorch and Why Choose It? . . . 10
3.2 Tensors: The Heart of PyTorch . . . 11
3.2.1 What Are Tensors? . . . 11
3.2.2 Why We Need Tensors in Machine Learning . . . 11
3.3 Data Generation . . . 12
3.3.1 Code Implementation . . . 12
3.3.2 Explanation . . . 12
3.3.3 Visualization . . . 12
3.4 Data Splitting . . . 12
3.4.1 Code Implementation . . . 12
3.4.2 Explanation . . . 13
3.5 Model Definition . . . 13
3.5.1 Code Implementation . . . 13
3.5.2 Explanation . . . 14
3.6 Loss Function and Optimizer . . . 14
3.6.1 Code Implementation . . . 14
3.6.2 Explanation . . . 14
3.7 Training Loop . . . 15
3.7.1 Code Implementation . . . 15
3.7.2 Explanation of training steps . . . 16
3.8 Model Evaluation . . . 16
3.8.1 Code Implementation . . . 16
3.8.2 Explanation . . . 17
3.9 Making Predictions . . . 17
3.9.1 Code Implementation . . . 17
3.9.2 Explanation . . . 17
3.10 Key Concepts Summary . . . 18

4 Common Issues and Fixes **18**

5 Non-linear regression — the same idea, plus activation functions **18**
5.1 Why add activation functions? . . . 18
5.2 Common activation functions (quick reference) . . . 19
5.3 One-layer example in PyTorch (ReLU) . . . 19
5.4 Alternative: using `tanh` or `sigmoid` . . . 19
5.5 Intuition and when to use . . . 20

6 Classification (overview) **20**

7 Binary classification (detailed) **20**
7.1 Problem vs regression . . . 20
7.2 Loss function: binary cross-entropy (log loss) . . . 21
7.3 Intuition behind the logarithm . . . 21
7.3.1 Concrete numeric examples (step-by-step) . . . 21
7.4 Why this matches probability and maximum likelihood . . . 22
7.5 PyTorch practical note . . . 22

8 Multiclass classification (brief) **23**

9 Quick summary **23**

10 Conclusion and Next Steps **23**

Course Overview

Welcome to AI DZ Academy's Flagship Machine Learning Course!

This course is designed to transform you from a beginner to a competent machine learning practitioner. We combine rigorous mathematical foundations with hands-on implementation.

Learning Objectives

- **Understand** the mathematical foundations of machine learning algorithms
- **Implement** supervised, unsupervised, and reinforcement learning models
- **Master** PyTorch for deep learning and neural networks
- **Develop** skills in data preprocessing and feature engineering
- **Learn** model evaluation, hyperparameter tuning, and deployment strategies
- **Build** a portfolio of machine learning projects

Course Structure

Week	Module	Topics	Projects
1	Foundations	ML Types, Math Review, Python Basics	Setup Development Environment
2	Linear Models	Linear/Logistic Regression, Optimization	House Price Prediction
3	Neural Networks	Perceptrons, Activation Functions, Backpropagation	Digit Recognition
4	Deep Learning	CNNs, RNNs, Transformers	Image Classification
5	Unsupervised Learning	Clustering, Dimensionality Reduction	Customer Segmentation
6	Advanced Topics	Reinforcement Learning, GANs	Game AI Agent
7	Production ML	Model Deployment, MLOps, Ethics	Deploy Model to Cloud
8	Capstone	End-to-End ML Project	Final Project Presentation

Table 1: Course Structure and Timeline

1 The Machine Learning Landscape

1.1 What is Machine Learning?

Definition 1.1 (Machine Learning). *Machine Learning (ML) is when computers learn from examples and experience, instead of being told exactly what to do. It's like teaching a child by showing them many pictures of cats and dogs until they can recognize the difference themselves.*

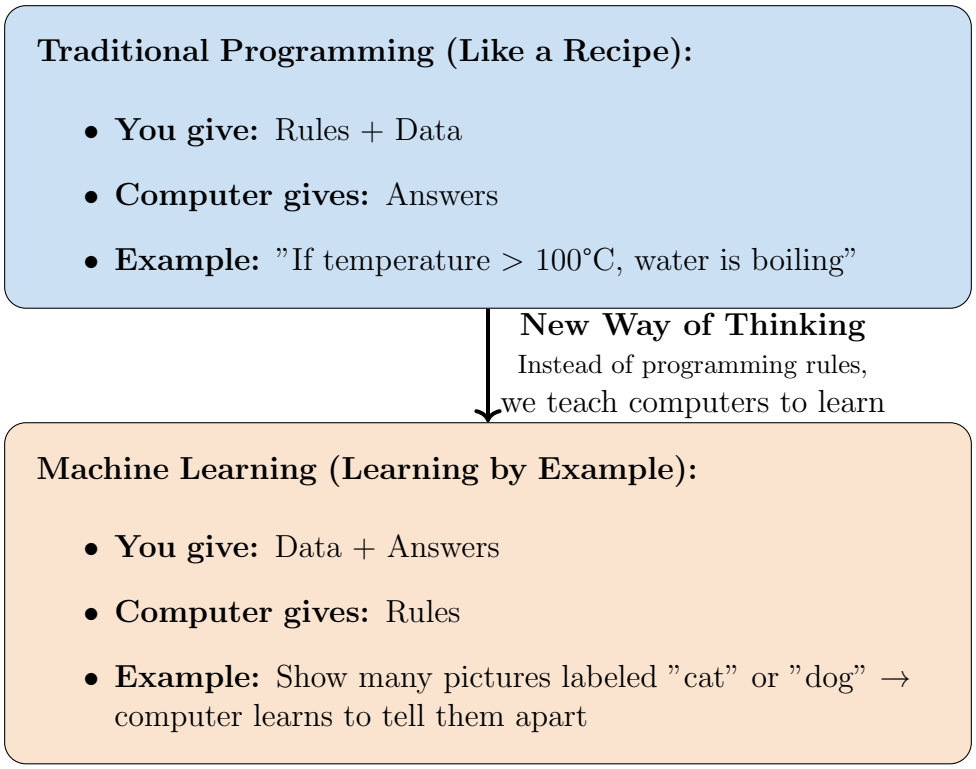


Figure 1: Difference Between Traditional Programming and Machine Learning

1.2 A Brief History of Machine Learning

1.3 Where Do We Use Machine Learning?

1.3.1 Real Examples That Help People

Example 1.1 (Helping Doctors). ***Problem:*** *It takes years to train eye doctors to spot eye diseases*
ML Solution: *Show a computer thousands of eye scans to learn patterns*
Result: *Computer can help doctors find problems faster, especially in remote villages*

Example 1.2 (Keeping Your Money Safe). ***Problem:*** *Banks can't check every transaction for fraud manually*
ML Solution: *Teach computer what normal vs. suspicious transactions look like*
Result: *Your card gets blocked automatically if someone tries to use it fraudulently*

Decade	Big Idea	What It Made Possible
1950s	First learning machine (Perceptron)	Simple pattern recognition, like telling left from right
1980s	Backpropagation (learning from mistakes)	Training smarter networks with multiple layers
1990s	Support Vector Machines	Better at classifying things, like spam vs not spam
2000s	Deep Learning	Computers that can "see" (image recognition) and "hear" (speech recognition)
2010s	Big Data + Powerful GPUs	Much faster learning with huge amounts of data
2020s	Transformers (like GPT)	Amazing language understanding, chatbots, translation

Table 2: Machine Learning Through the Years

2 The Three Main Types of Machine Learning

2.1 Supervised Learning: Learning with a Teacher

Definition 2.1 (Supervised Learning)**.** *Learning with labeled examples. Like studying with answer keys! You give the computer questions WITH answers, and it learns to answer new questions.*

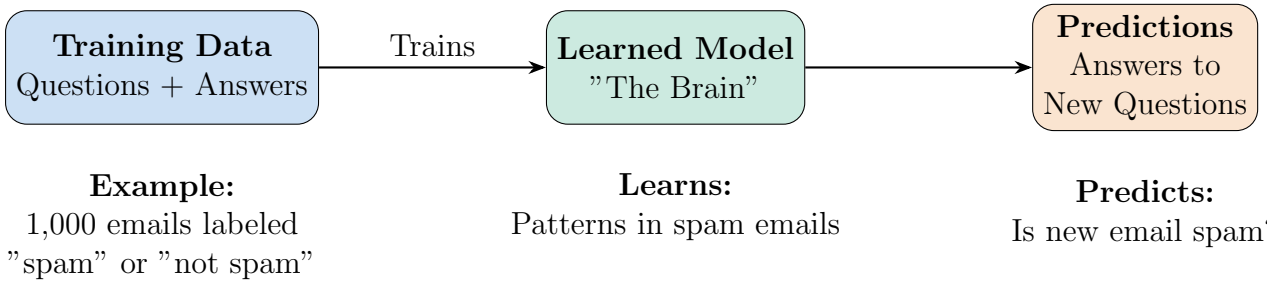


Figure 3: Supervised Learning Process

2.1.1 Common Tasks in Supervised Learning

2.2 Unsupervised Learning: Finding Hidden Patterns

Definition 2.2 (Unsupervised Learning)**.** *Learning without labels. Like exploring a new city without a map! The computer finds patterns and groups in data all by itself.*

Example 2.1 (Organizing Your Music)**.** ***Problem:*** *You have 5,000 songs with no playlist* ***ML Solution:*** *Computer listens to all songs and groups them by similarity*

Predicting Numbers (Regression)	Picking Categories (Classification)	Complex Outputs
• **Goal:** Predict a number • **Examples:** – House price – Temperature tomorrow – Your test score	• **Goal:** Choose a category • **Examples:** – Spam or not spam? – Cat or dog? – Happy or sad face?	• **Goal:** Generate complex output • **Examples:** – Write a sentence – Draw a picture – Translate languages

Table 3: Different Types of Supervised Learning Problems

Result: *Creates playlists like "Chill Lo-Fi", "Workout Mix", "Study Focus" automatically*

2.3 Reinforcement Learning: Learning by Trial and Error

Definition 2.3 (Reinforcement Learning)**.** *Learning by trying different actions and seeing what works best. Like learning to ride a bike - you try, sometimes fall (negative feedback), sometimes balance (positive feedback), and gradually improve.*

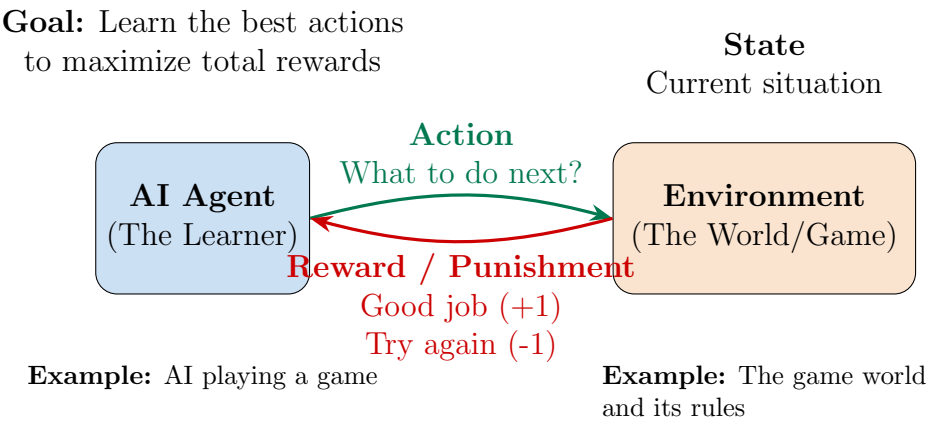


Figure 4: Reinforcement Learning: The Feedback Loop

Example 2.2 (Training a Game AI)**.** ***Problem:*** *How to make AI good at a video game?* ***RL Approach:*** *Let AI play millions of times, rewarding good moves and penalizing bad ones*
Result: *AI becomes superhuman at games like Go, Dota 2, or StarCraft*

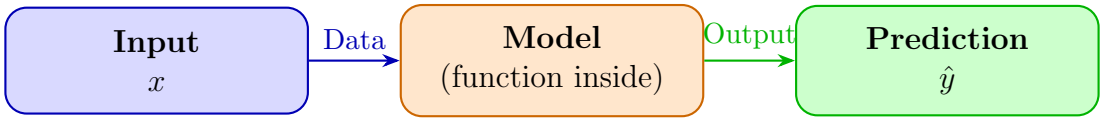
3 Deep Dive: Linear Regression

Before going deeper into training, let us clearly understand what a **model** is.

Model as a black box: input → output

At the highest level, a model can be seen as a simple system:

- It receives an **input** x
- It produces an **output** $\hat{y}$



At this stage, the model looks like a *black box*. The natural question is:

What is inside the model?

What is inside the model?

In a very simple manner, **there is nothing magical inside the model**. A model is just a **mathematical function**:

$$\hat{y} = f(x)$$

- The input is x
- The output is $\hat{y}$
- The function $f(\cdot)$ is chosen based on the **type of dataset**

Important idea: Different datasets ⇒ different types of functions.

Assumption: linear dataset

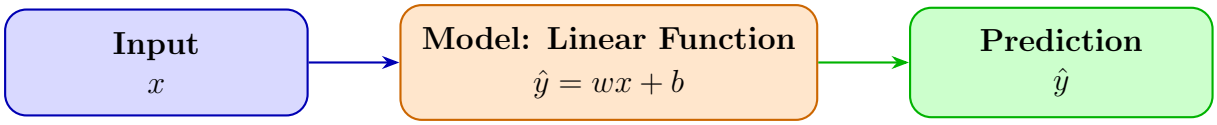
In this first section, we assume that our dataset is **linear**. This means the relationship between input x and output y can be approximated by a straight line.

So we choose a **linear function** for our model:

$$\hat{y} = wx + b$$

- w is the **weight** (slope of the line)
- b is the **bias** (vertical shift)

Now our model is no longer a mystery. Inside the box, we simply have:



Goal of linear regression

Our dataset contains many points (x, y). The model produces predictions $(x, \hat{y})$.

The main goal of linear regression is:

To find the best values of w and b such that the line

$$\hat{y} = wx + b$$

fits the dataset as well as possible.

In practice, this means:

- The predicted values $\hat{y}$ should be close to the true values y
- The distance (error) between the line and the data points should be as small as possible

This distance is measured using a **loss function**, and training consists of adjusting w and b to **minimize this distance**.

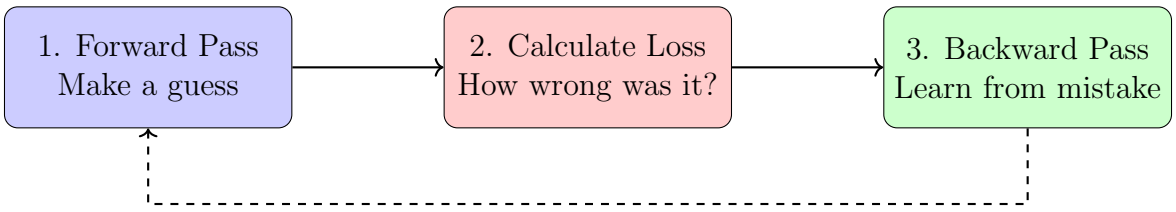
Intuition

- The model starts with random w and b
- The line is usually a bad fit at the beginning
- After many learning steps, the line slowly moves and rotates
- Eventually, it fits the data as closely as possible

Key takeaway: Linear regression is simply about finding the best straight line that explains the relationship between inputs and outputs by minimizing the distance between predictions and real data points.

2.4 The Three Magic Steps of Learning

Every time your model learns, it goes through three steps:



Below we explain each step in a simple and concrete way with a tiny math example.

1. Forward pass ("make a guess")

The model uses its current parameters (weights and biases, written collectively as θ) to produce a prediction for an input x. We write the model as a function

$$\hat{y} = f_\theta(x).$$

Example (very simple): a linear model with weight W and bias b:

$$\hat{y} = Wx + b.$$

Intuition: the forward pass is just running your input through the equations of the model to see what it predicts.

2. Calculate loss ("how wrong was it?")

We compare the model's prediction $\hat{y}$ to the true target y using a loss function $\mathcal{L}(\hat{y}, y)$. A common choice for regression is the mean squared error (MSE) for a single sample:

$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2.$$

Intuition: the loss is a single number that tells us how bad the guess was — bigger means worse.

Concrete numeric example (one sample). Suppose $x = 2$, $y = 3$, and initial parameters $W = 0.5$, $b = 0$.

$$\text{Prediction: } \hat{y} = Wx + b = 0.5 \times 2 + 0 = 1.$$
$$\text{Loss: } \mathcal{L} = (\hat{y} - y)^2 = (1 - 3)^2 = 4.$$

So the model's guess is far from the true value and the loss equals 4.

3. Backward pass ("learn from the mistake")

The backward pass computes how to change the parameters θ to reduce the loss. Concretely, we compute the gradient (partial derivatives) of the loss with respect to each parameter:

$$\nabla_\theta \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \theta_1}, \frac{\partial \mathcal{L}}{\partial \theta_2}, \dots \right).$$

For the simple linear model $\hat{y} = Wx + b$ and loss $\mathcal{L} = (\hat{y} - y)^2$, the derivatives for one sample are:

$$\frac{\partial \mathcal{L}}{\partial W} = 2(\hat{y} - y)\,\frac{\partial \hat{y}}{\partial W} = 2(\hat{y} - y)\,x,$$
$$\frac{\partial \mathcal{L}}{\partial b} = 2(\hat{y} - y)\,\frac{\partial \hat{y}}{\partial b} = 2(\hat{y} - y).$$

Continue the numeric example. Using the numbers above:

$$\hat{y} - y = 1 - 3 = -2.$$

So

$$\frac{\partial \mathcal{L}}{\partial W} = 2(-2) \cdot 2 = -8, \qquad \frac{\partial \mathcal{L}}{\partial b} = 2(-2) = -4.$$

These gradients tell us which direction to change each parameter to reduce loss.

Parameter update (gradient descent)

A simple update rule is gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L},$$

where $\eta > 0$ is the learning rate (a small step size). Using $\eta = 0.1$ in our example:

$$W \leftarrow 0.5 - 0.1 \times (-8) = 0.5 + 0.8 = 1.3, \qquad b \leftarrow 0 - 0.1 \times (-4) = 0.4.$$

After this update the model will predict a value closer to $y = 3$ for the same input $x = 2$.

Putting it all together (one training step)

1. **Forward:** compute $\hat{y} = f_\theta(x)$.
2. **Loss:** compute $\mathcal{L}(\hat{y}, y)$.
3. **Backward:** compute gradients $\nabla_\theta \mathcal{L}$.
4. **Update:** $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$.

Notes and intuition

- **Why gradients?** Gradients point in the direction of greatest increase of the loss. Subtracting a small multiple of the gradient moves parameters in the direction that reduces the loss.
- **Batch vs single sample:** In practice we usually compute the average loss and average gradients over a small set of examples (a *mini-batch*) rather than a single sample. This smooths updates and often helps training.
- **Learning rate η:** too large $\rightarrow$ unstable training; too small $\rightarrow$ very slow learning. It is a hyperparameter you tune.
- **Backpropagation:** for deep neural networks the backward pass uses the chain rule repeatedly to compute gradients layer-by-layer. Frameworks (PyTorch, TensorFlow) compute these gradients automatically.

Final thought: Think of the model as a student taking a quiz. The forward pass is the student answering the question, the loss is the teacher grading how wrong the answer is, and the backward pass is the student studying the feedback and adjusting how they think so the next answer is better.

3 PyTorch Mastery: From Fundamentals to First Model

3.1 What is PyTorch and Why Choose It?

Definition 3.1 (PyTorch)**.** *PyTorch is an open-source machine learning framework that provides two main features:*

- ***Tensor computation*** *with strong GPU acceleration*
- ***Deep neural networks*** *built on a tape-based autograd system*

Why PyTorch is beginner-friendly:

1. **Pythonic nature**: Feels like writing regular Python code
2. **Dynamic computation graph**: Build and modify networks on-the-fly
3. **Easy debugging**: Use standard Python debugging tools
4. **Strong community**: Extensive tutorials and documentation
5. **Research-friendly**: Widely used in academia and industry

```
# Comparison with TensorFlow (simplified)
# PyTorch - More intuitive, Pythonic
model(x)  # Just call the model like a function

# TensorFlow - More declarative
model.predict(x)  # Special method needed
```

Listing 1: Why PyTorch over other frameworks

3.2 Tensors: The Heart of PyTorch

3.2.1 What Are Tensors?

Analogie

Think of tensors as **containers for data**:

- **Scalar** (0D tensor): A single number → Temperature: 25°C
- **Vector** (1D tensor): List of numbers → Daily temperatures: [25, 26, 24]
- **Matrix** (2D tensor): Table of numbers → Temperature grid: [[25, 26], [24, 25]]
- **3D+ Tensor**: Higher dimensions → Video data: [frames × height × width × colors]

3.2.2 Why We Need Tensors in Machine Learning

- **Efficient computation**: Specialized operations optimized for speed
- **GPU acceleration**: Move computations to GPU for 50–100× speedup
- **Automatic differentiation**: Track operations for gradient calculation
- **Standardized format**: All ML libraries understand tensor format

3.3 Data Generation

3.3.1 Code Implementation

```
import torch
import numpy as np
import matplotlib.pyplot as plt

# Define true parameters (unknown to the model initially)
a = 0.3  # True slope
b = 0.7  # True intercept

# Create input data: 50 points from 0 to 0.98
X = torch.arange(0., 1., 0.02).float()  # shape (50,)

# Generate target values with linear relationship (no noise here)
y = a * X + b

print(f"X = {X}\n\ny = {y}")
```

Listing 2: Generating synthetic data

3.3.2 Explanation

We create synthetic data following a linear relationship $y = 0.3x + 0.7$. This approach allows us to know the "ground truth" and evaluate our model's performance accurately.

3.3.3 Visualization

```
plt.scatter(X.numpy(), y.numpy())
plt.xlabel('X')
plt.ylabel('y')
plt.title('Synthetic Linear Data')
plt.show()
```

Listing 3: Visualizing the data

3.4 Data Splitting

3.4.1 Code Implementation

```
# 80% for training, 20% for testing
split = int(0.8 * len(X))
X_train, y_train = X[:split], y[:split]
X_test, y_test = X[split:], y[split:]

plt.scatter(X_train.numpy(), y_train.numpy(), label='Training data')
plt.scatter(X_test.numpy(), y_test.numpy(), c='red', label='Test data')
plt.xlabel('X')
plt.ylabel('y')
plt.title('Training vs Test Data Split')
plt.legend()
plt.show()
```

Listing 4: Splitting data into training and test sets

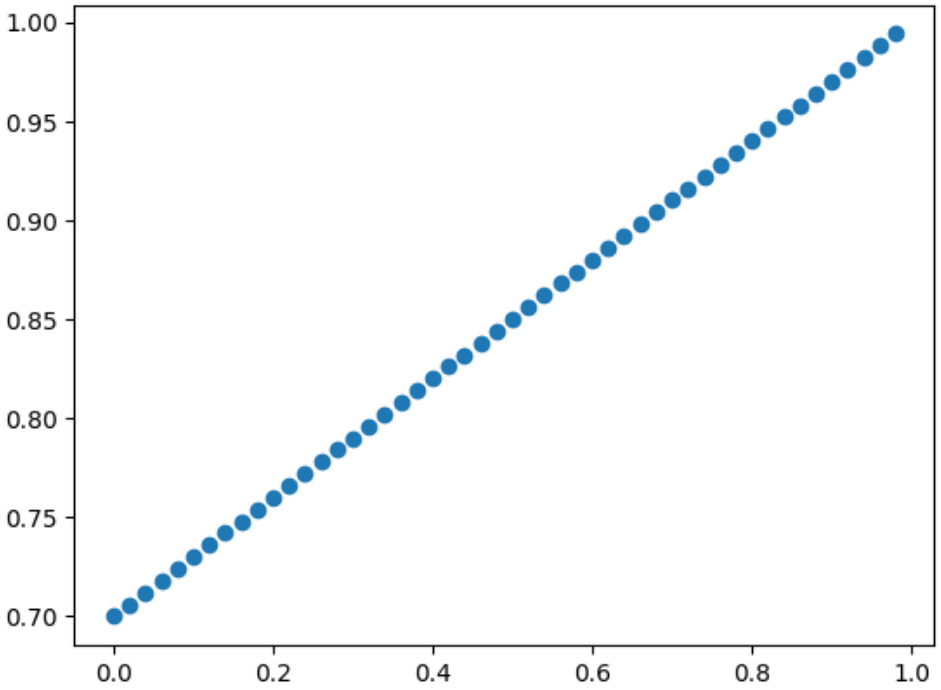


Figure 5: Visual representation of our synthetic linear data

3.4.2 Explanation

We split our data into training (40 points) and test (10 points) sets. This allows us to:

- Train the model on the training set
- Evaluate its performance on unseen test data
- Check for overfitting

3.5 Model Definition

3.5.1 Code Implementation

```
import torch.nn as nn
torch.manual_seed(43)  # For reproducibility

class LinearRegressionModel(nn.Module):
    def __init__(self):
        super().__init__()
        # Initialize parameters (single weight and bias)
        # Using nn.Parameter so they are registered and optimized
        self.w = nn.Parameter(torch.randn(1, dtype=torch.float32))
        self.b = nn.Parameter(torch.randn(1, dtype=torch.float32))

    def forward(self, x):
        # x is expected to be a 1-D tensor (shape: [N,])
        return self.w * x + self.b

model = LinearRegressionModel()
```

Listing 5: Defining the linear regression model

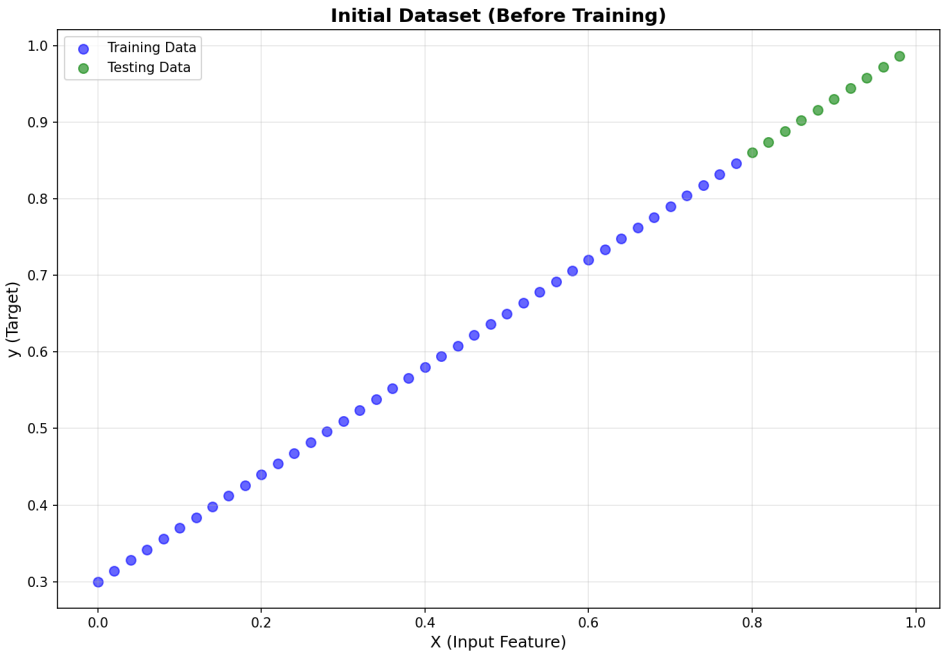


Figure 6: Visual representation of our training and test datasets

3.5.2 Explanation

- `nn.Module`: Base class for all neural network modules.
- `nn.Parameter`: Tensors that will be optimized during training.
- `requires_grad`: default for `nn.Parameter` is True, so gradients will be computed.
- `forward()`: Defines the computation performed at every call.
- **Note**: No activation (ReLU) is used here because we want a true linear model.

3.6 Loss Function and Optimizer

3.6.1 Code Implementation

```
# Mean Absolute Error (L1 Loss) or use MSELoss for mean squared error
criterion = nn.L1Loss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

Listing 6: Setting up loss function and optimizer

3.6.2 Explanation

The loss function measures the discrepancy between predictions and targets. The optimizer updates the model parameters using gradients computed from the loss. For gradient-descent-style updates:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta)$$

where η is the learning rate.

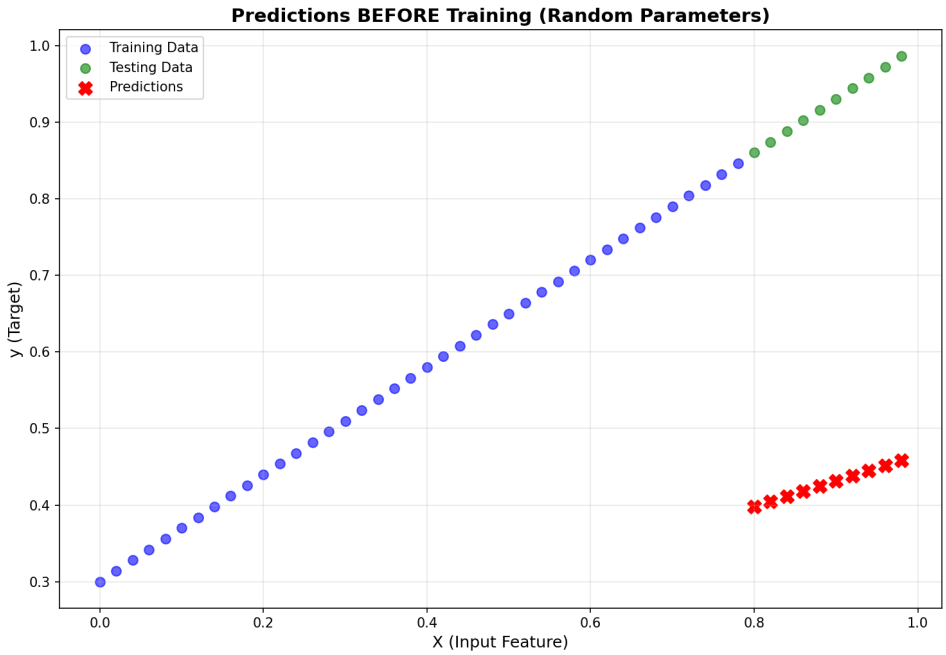


Figure 7: Visual representation of the model predection befor training

3.7 Training Loop

3.7.1 Code Implementation

```
epochs = 250
loss_history = []

for epoch in range(epochs):
    model.train()  # Set model to training mode

    # Forward pass: compute predictions on training set
    y_pred = model(X_train)

    # Compute loss
    loss = criterion(y_pred, y_train)
    loss_history.append(loss.item())

    # Backward pass and optimization
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

# Plot loss curve
import matplotlib.pyplot as plt
plt.plot(loss_history)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss over Epochs')
plt.show()
```

Listing 7: Training the model

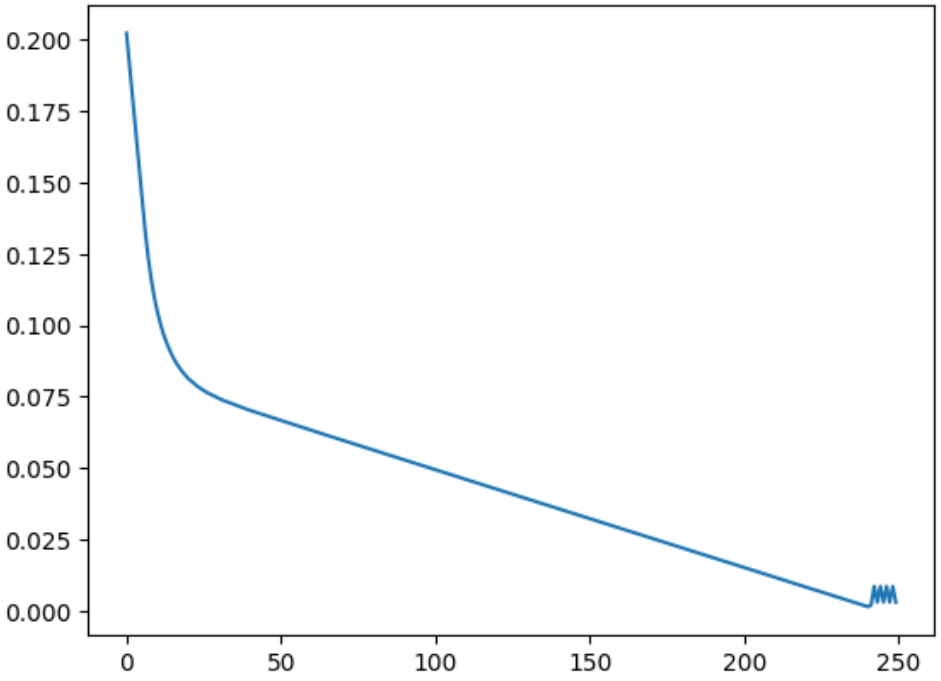


Figure 8: Training Loss over Epochs

3.7.2 Explanation of training steps

1. **Forward Pass**: Compute predictions $\hat{y} = f_\theta(X)$.
2. **Loss Computation**: Calculate error between predictions and true values.
3. **Backward Pass**: Compute gradients $\nabla_\theta J$ using backpropagation.
4. **Parameter Update**: Adjust parameters in the direction that minimizes loss.

3.8 Model Evaluation

3.8.1 Code Implementation

```
model.eval()  # Set model to evaluation mode
with torch.no_grad():  # Disable gradient computation for inference
    y_test_pred = model(X_test)

# Plot training data, true test values and predictions
plt.scatter(X_train.numpy(), y_train.numpy(), label='Training data')
plt.scatter(X_test.numpy(), y_test.numpy(), c='green', label='True test
    values')
plt.scatter(X_test.numpy(), y_test_pred.numpy(), c='red', label='
   Predictions')
plt.xlabel('X')
plt.ylabel('y')
plt.title('Model Predictions vs True Values')
plt.legend()
plt.show()
```

Listing 8: Evaluating on the test set

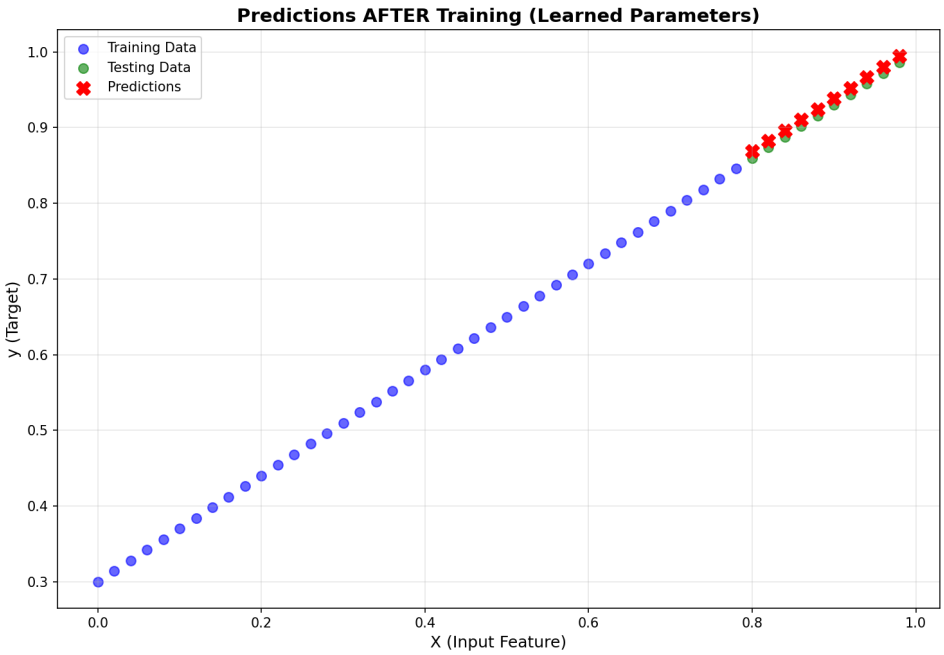


Figure 9: Model Predictions after training vs True Values

3.8.2 Explanation

- `model.eval()`: Sets the model to evaluation mode (affects layers like dropout or batchnorm).
- `torch.no_grad()`: Disables gradient computation to save memory during inference.
- The plot shows how well the model generalizes to unseen data.

3.9 Making Predictions

3.9.1 Code Implementation

```
# Example single prediction
x = 0.79
y_true = a * x + b  # True value using known parameters
y_pred_single = model(torch.tensor([x], dtype=torch.float32))

print(f"True value: y = {y_true:.3f}")
print(f"Predicted value: y_hat = {y_pred_single.item():.3f}")
```

Listing 9: Making a single prediction

3.9.2 Explanation

For $x = 0.79$:

- True value: $y = 0.3 \times 0.79 + 0.7 \approx 0.937$
- Model prediction: shown by `y_pred_single`
- A small difference indicates successful learning

3.10 Key Concepts Summary

Concept	Explanation
Parameters (w, b)	Learned weight and bias of the linear model
Forward Pass	Computing predictions from inputs
Loss Function	Measures error between predictions and true values
Backward Pass	Computing gradients via backpropagation
Optimizer	Updates parameters to minimize loss
Training Loop	Iterative forward, loss, backward and optimization steps
Model Evaluation	Testing on unseen data to assess generalization

Table 4: Key concepts in linear regression with PyTorch

4 Common Issues and Fixes

- **Learning rate**: Too high causes divergence; too low causes slow convergence.
- **Data normalization**: Not needed for this synthetic example but important for real-world datasets.
- **Gradient accumulation**: Always call `optimizer.zero_grad()` before `loss.backward()`.

5 Non-linear regression — the same idea, plus activation functions

A **non-linear regression** model is conceptually the same as linear regression, but we insert *activation functions* (nonlinear functions) into the model. Where linear regression uses a single linear map

$$\hat{y} = wx + b,$$

a non-linear regression model applies a nonlinear function $\sigma(\cdot)$ to the linear combination:

$$\hat{y} = \sigma(wx + b).$$

Because σ is non-linear, the mapping from x to $\hat{y}$ can represent much richer relationships than a straight line.

5.1 Why add activation functions?

- **Introduce nonlinearity:** Without an activation, a stack of linear layers is still equivalent to a single linear function. Activations let the model learn curved, piecewise or more complex shapes.
- **Model expressivity:** With appropriate activations, even small networks can approximate complicated functions (this is the intuition behind universal approximation results).
- **Control output range:** Some activations (e.g. tanh, sigmoid) limit the output range, which can be useful depending on the target variable.

5.2 Common activation functions (quick reference)

ReLU: $\mathrm{ReLU}(z) = \max(0, z)$ (simple, fast, piecewise linear)

Leaky ReLU: $\mathrm{LeakyReLU}(z) = \begin{cases} z & z \geq 0 \\ \alpha z & z < 0 \end{cases} \quad (\alpha > 0)$

Sigmoid: $\sigma(z) = \dfrac{1}{1 + e^{-z}}$ (maps to $(0, 1)$)

Tanh: $\tanh(z) = \dfrac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$ (maps to $(-1, 1)$)

Important note for regression: if your target y is unbounded and real-valued, you typically *do not* force the final layer through a bounded activation like sigmoid — instead you use nonlinear activations in the hidden layers and keep the final output linear (identity). In the simple single-layer example below we show the activation applied to the linear output to illustrate the idea; for many regression tasks you might prefer $\hat{y} = wx + b$ as the final step and place activations only if you add hidden layers.

5.3 One-layer example in PyTorch (ReLU)

Below is a corrected, working PyTorch example that shows how to add an activation to a simple linear model.

```
import torch
import torch.nn as nn

torch.manual_seed(43)

class NonLinearRegressionModel(nn.Module):
    def __init__(self):
        super().__init__()
        # single weight and bias (scalar)
        self.w = nn.Parameter(torch.randn(1, dtype=torch.float32))
        self.b = nn.Parameter(torch.randn(1, dtype=torch.float32))
        # activation (nonlinearity)
        self.relu = nn.ReLU()

    def forward(self, x):
        # ensure x is a float tensor; broadcasting works for scalar w, b
        return self.relu(self.w * x + self.b)

# usage example
model = NonLinearRegressionModel()
x = torch.tensor([0.1, 0.5, 1.0], dtype=torch.float32)
y_hat = model(x)  # y_hat is nonlinearly related to x
print(y_hat)
```

Listing 10: Single-layer model with ReLU activation

5.4 Alternative: using `tanh` or `sigmoid`

If you want a smooth nonlinear transformation (not piecewise linear) you can replace `nn.ReLU()` by `nn.Tanh()` or `nn.Sigmoid()`:

```
# replace activation in the model:
self.activation = nn.Tanh()        # or nn.Sigmoid(), nn.LeakyReLU(0.01), ...
# forward: return self.activation(self.w * x + self.b)
```

Listing 11: Switching activation function

5.5 Intuition and when to use

- If data are approximately linear: a plain linear model may suffice and is easier to interpret.
- If data show curves, saturation, or piecewise behaviour: add nonlinearities (activations) and/or hidden layers.
- For simple tasks, a single linear layer + activation can already model piecewise linear relationships (ReLU) — but deeper or wider networks increase flexibility.
- For regression to unbounded real targets, prefer **no final bounding activation** (keep final output linear) and place activations in hidden layers.

6 Classification (overview)

Classification is the task of assigning discrete labels to inputs. There are two common families:

- **Binary classification:** only two possible classes (often denoted 0 and 1). Example: spam vs not-spam.
- **Multiclass classification:** more than two classes and each example belongs to exactly one class (single-label). Example: digit recognition (0–9).

The main difference: in binary classification we predict a single probability ("is this class 1?"); in multiclass we predict a probability distribution across $K > 2$ classes.

7 Binary classification (detailed)

7.1 Problem vs regression

If you try to use a plain linear regression model for a binary problem you get real-valued outputs that are not probabilities. We need outputs in the range $[0, 1]$ that can be interpreted as $\Pr(\text{class} = 1 \mid x)$. To obtain probabilities we apply a **sigmoid** activation to the linear output (logit).

$$z = wx + b \quad \text{(logit)}$$

$$\hat{p} \equiv \hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

Now $\hat{y}$ is a probability: large positive $z \Rightarrow \hat{y} \approx 1$; large negative $z \Rightarrow \hat{y} \approx 0$.

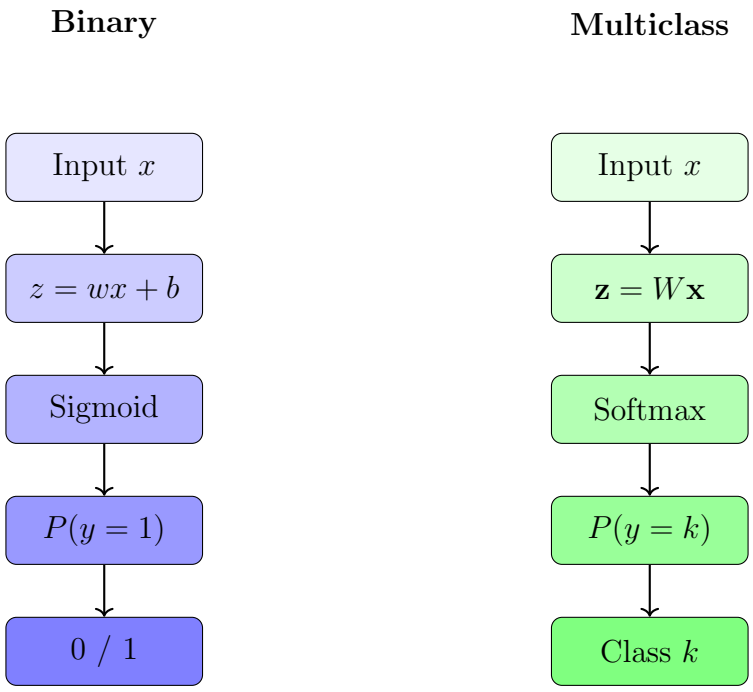


Figure 10: Binary vs multiclass classification (simplified view)

7.2 Loss function: binary cross-entropy (log loss)

For a single sample with true label $y \in \{0, 1\}$ and predicted probability $\hat{y} \in (0, 1)$, the binary cross-entropy loss (also called log loss) is

$$J(\hat{y}, y) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right].$$

This formula does two things in one expression:

- If $y = 1$, the loss reduces to $J = -\log(\hat{y})$.
- If $y = 0$, the loss reduces to $J = -\log(1 - \hat{y})$.

7.3 Intuition behind the logarithm

- The logarithm makes the loss *penalize confident wrong predictions very heavily.* Because $\log(z) \to -\infty$ when $z \to 0^+$, if the model assigns a probability near 0 to the true class the loss becomes very large.
- Conversely, if the model assigns a probability close to 1 to the true class, $-\log(\hat{y})$ is small, so the loss is small.

7.3.1 Concrete numeric examples (step-by-step)

Case A: true label $y = 1$, good prediction $\hat{y} = 0.99$.

$$J = -\log(0.99)$$

Compute (natural log):

$$\log(0.99) \approx -0.0100503 \quad \Rightarrow \quad J \approx 0.01005$$

So a confident correct prediction yields a very small loss.

Case B: true label $y = 1$, bad prediction $\hat{y} = 0.01$.

$$J = -\log(0.01)$$

$$\log(0.01) = \log(10^{-2}) = -2\log(10) \approx -4.60517 \quad \Rightarrow \quad J \approx 4.60517$$

So a confident wrong prediction produces a very large loss.

Symmetry for $y = 0$. If $y = 0$, the loss is $J = -\log(1 - \hat{y})$.

- If $\hat{y} = 0.01$: $J = -\log(0.99) \approx 0.01005$ (small loss)
- If $\hat{y} = 0.99$: $J = -\log(0.01) \approx 4.60517$ (large loss)

This symmetry is why a single formula covers both cases.

7.4 Why this matches probability and maximum likelihood

Binary cross-entropy is (up to scaling) the negative log-likelihood of a Bernoulli model. Minimizing the average of this loss across the dataset is equivalent to maximum likelihood estimation.

7.5 PyTorch practical note

For numerical stability you should usually feed the raw linear output (logits) into `nn.BCEWithLogitsLoss()` rather than applying `sigmoid` yourself and using `nn.BCELoss()`. `BCEWithLogitsLoss` combines both in a stable way.

```python
import torch
import torch.nn as nn

torch.manual_seed(43)

class BinaryLinearModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.w = nn.Parameter(torch.randn(1))
        self.b = nn.Parameter(torch.randn(1))

    def forward(self, x):
        return self.w * x + self.b

model = BinaryLinearModel()
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# training loop (sketch)
logits = model(X)
loss = criterion(logits, y)  # y in {0.0, 1.0}
loss.backward()
optimizer.step()
```

Listing 12: Binary model with logits and stable loss in PyTorch

For inference:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Then convert probability to a class using a threshold (typically 0.5):

$$\hat{c} = \begin{cases} 1 & \text{if } \hat{y} \geq 0.5 \\ 0 & \text{if } \hat{y} < 0.5 \end{cases}$$

8 Multiclass classification (brief)

- For single-label multiclass classification we use a **softmax** layer:

$$\hat{p}_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \quad i = 1, \ldots, K$$

- The loss is categorical cross-entropy (PyTorch: `nn.CrossEntropyLoss`). It expects raw logits and integer labels $0, \ldots, K-1$.

9 Quick summary

- **Binary classification:** sigmoid + binary cross-entropy (`BCEWithLogitsLoss`)
- **Multiclass classification:** softmax + categorical cross-entropy (`CrossEntropyLoss`)
- **Log intuition:** small loss when confidently correct, huge loss when confidently wrong

10 Conclusion and Next Steps

Congratulations!

You've mastered the first foundational module
of AI DZ Academy's Machine Learning Course.

You now have a solid understanding of:
Linear Regression & PyTorch Fundamentals

Next Module:

Neural Networks & Deep Learning

- Perceptrons
- Activations
- Backpropagation
- CNNs
- RNNs

Ready to build intelligent systems? The journey continues...

Additional Resources

Books:

- *Pattern Recognition and Machine Learning* - Christopher Bishop
- *Deep Learning* - Ian Goodfellow, Yoshua Bengio, Aaron Courville
- *Hands-On Machine Learning* - Aurélien Géron

Online Courses:

- Stanford CS229: Machine Learning (Andrew Ng)
- Fast.ai: Practical Deep Learning for Coders
- DeepLearning.AI Specializations on Coursera

Research Papers:

- "Attention Is All You Need" - Vaswani et al. (2017)
- "ImageNet Classification with Deep CNNs" - Krizhevsky et al. (2012)
- "Playing Atari with Deep RL" - Mnih et al. (2013)

Practical Tools:

- **MLflow**: Machine Learning Lifecycle Management
- **DVC**: Data Version Control
- **Weights & Biases**: Experiment Tracking
- **Gradio**: Model Deployment Interface

AI DZ ACADEMY

Empowering the Next Generation of AI Engineers

Contact: aidzacademy@gmail.com — whatsapp: +213658597060

Instagram:/ai_dz_academy/

Website: www.aidzacademy.org

Algiers, Algeria — © 2025 AI DZ Academy